

iOS Seminar 4

Wafflestudio 2025



Struct vs Class

Struct

- 값 타입 (Value Type)
- 변수에 할당되거나 함수에 인자로 전달되는 경우
새 인스턴스가 복사된다
- 메소드를 이용해 프로퍼티를 변경할 수 없다
 - mutating func 이용 필요

ex. SwiftUI의 View에서 var, let으로 선언된
변수는 변경 불가

Class

- 참조 타입 (Reference Type)
- 변수에 할당되거나 함수에 인자로 전달되는 경우
동일한 인스턴스에 대한 참조가 공유된다
 - (당연히) 변경사항이 모두에게 반영된다
- 메소드를 이용해 프로퍼티를 변경할 수 있다

ex. UIKit의 UIViewController에서 값 변경

```
navigationController?.title = "홈"
```


@State + Struct / Class

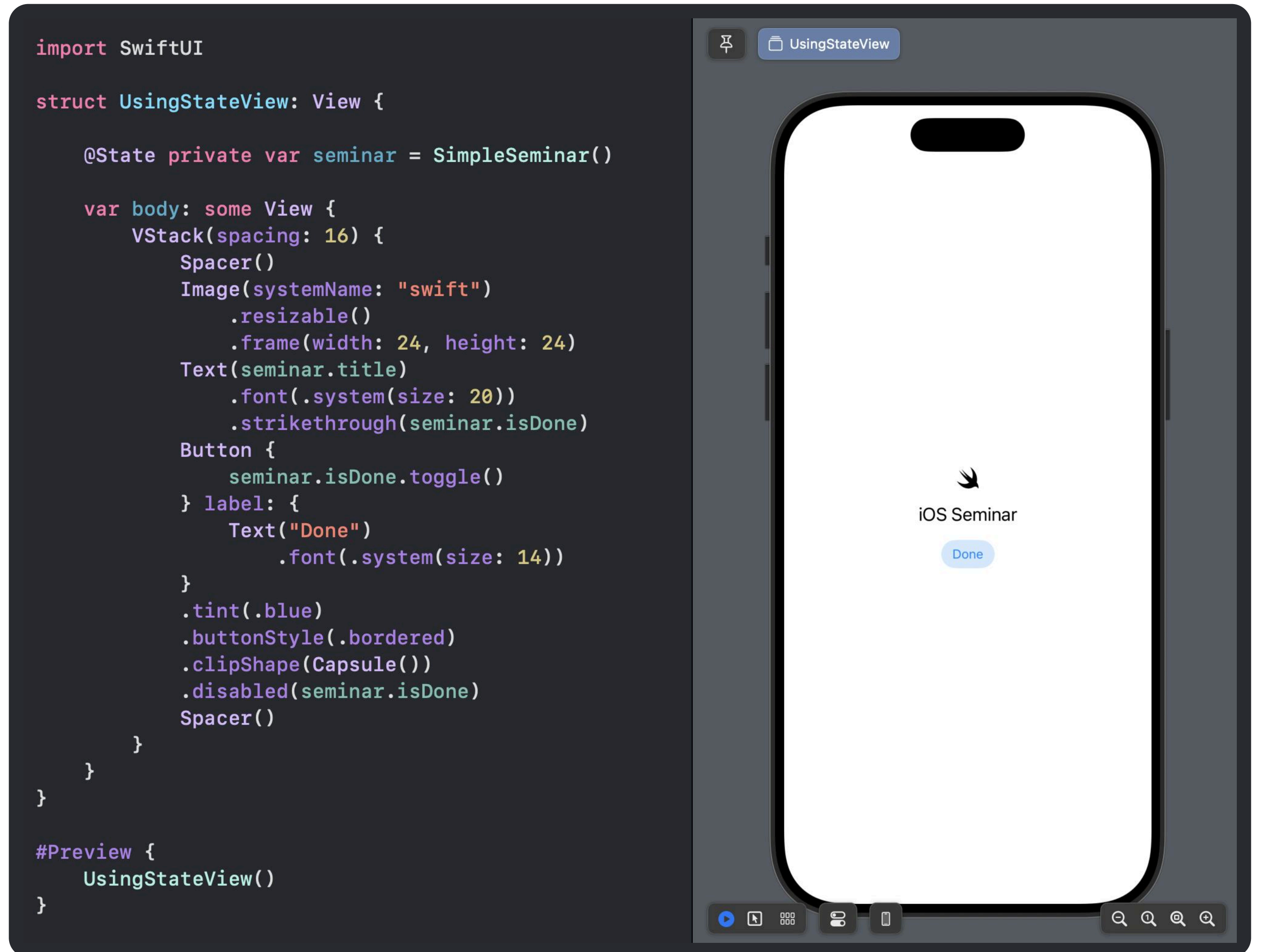
- SimpleSeminar (Class)의 isDone이 바뀌어도 UsingStateView는 업데이트되지 않는다

```
struct SimpleSeminar {  
    var title = "iOS Seminar"  
    var isDone = false  
}
```

Struct를 이용하는 경우

```
final class SimpleSeminar {  
    var title = "iOS Seminar"  
    var isDone = false  
}
```

Class를 이용하는 경우



UsingStateView.swift

정리하자면...

- Int, String 등의 Primitive Type 혹은 Struct인 값이 변경되면
- 그들을 들고 있는 SwiftUI View(Struct)도 빈번하게 새로 그려진다 (init)
- 문제: A의 값이 변경되어서 View를 새로 그리는 과정에서 B, C의 변경사항도 날아감
→ @State, @Binding 등을 이용해 방지
- Class인 값이 변경되면
- 그 값을 들고 있는 SwiftUI View(Struct)의 입장에서는 참조값이 바뀌지 않았으므로, 변경사항을 인지하지 못한다
- 문제: A의 값이 변경되었는데, View Update가 일어나지 않음
→ ObservableObject, @Observable 등 사용 (다음 장에 이어서)

SwiftUI+Data Managing (2)



SwiftUI가 View를 업데이트하는 방식 (공식 문서)

1. View State 공유 (Seminar 3에서 다룬 내용)

ex. @State @Binding 등...

2. Model Data의 변경사항 관찰

iOS 13.0+에서의 Model data Observation

- Combine(Framework) 이용
 - Publisher와 Subscriber가 있고, 한 개 이상의 Publisher Chain 끝에 Subscriber 존재
 - Publisher: `send()`
 - Subscriber: `sink(receiveCompletion:receiveValue:)` 혹은 `assign(to:on:)`
- Custom Publisher를 만드는 방법
 - (1) PassthroughSubject
 - (2) CurrentValueSubject
 - (3) (Class 내부 변수 중) 변화를 알리고 싶은 변수에 `@Published` 이용

iOS 13.0+에서의 Model data Observation

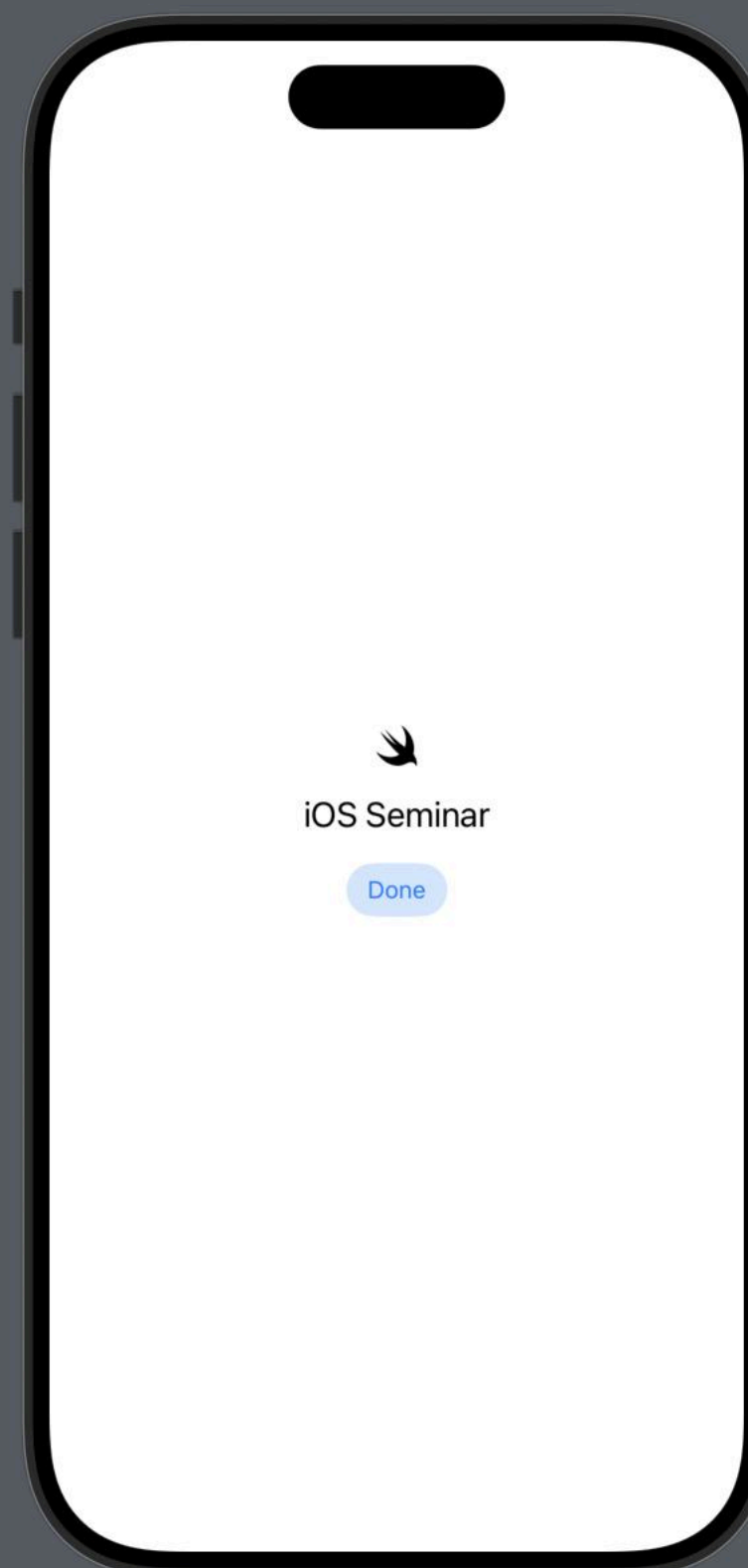
- `ObservableObject` 프로토콜 채택 (공식 문서)
 - 변화를 알리고 싶은 변수에 `@Published` 이용
 - @Published가 붙은 property는 변경되기 직전, 변화할 값을 방출 (Publisher 역할)
 - (1) Class 내부 프로퍼티의 변경사항을 알리거나
 - (2) 상위 publisher에서 받은 값을 외부로 re-publish하는 용도로도 활용 가능

@StateObject

```
import SwiftUI

struct PublishedExample: View {
    @StateObject var seminar = PublishingSeminar()

    var body: some View {
        VStack(spacing: 16) {
            Spacer()
            Image(systemName: "swift")
                .resizable()
                .frame(width: 24, height: 24)
            Text(seminar.title)
                .font(.system(size: 20))
                .strikethrough(seminar.isDone)
            Button {
                seminar.completeSeminar()
            } label: {
                Text("Done")
                    .font(.system(size: 14))
            }
                .tint(.blue)
                .buttonStyle(.bordered)
                .clipShape(Capsule())
                .disabled(seminar.isDone)
            Spacer()
        }
    }
}
```



```
import Foundation
import Combine

final class PublishingSeminar: ObservableObject {
    let title = "iOS Seminar"
    @Published var isDone = false

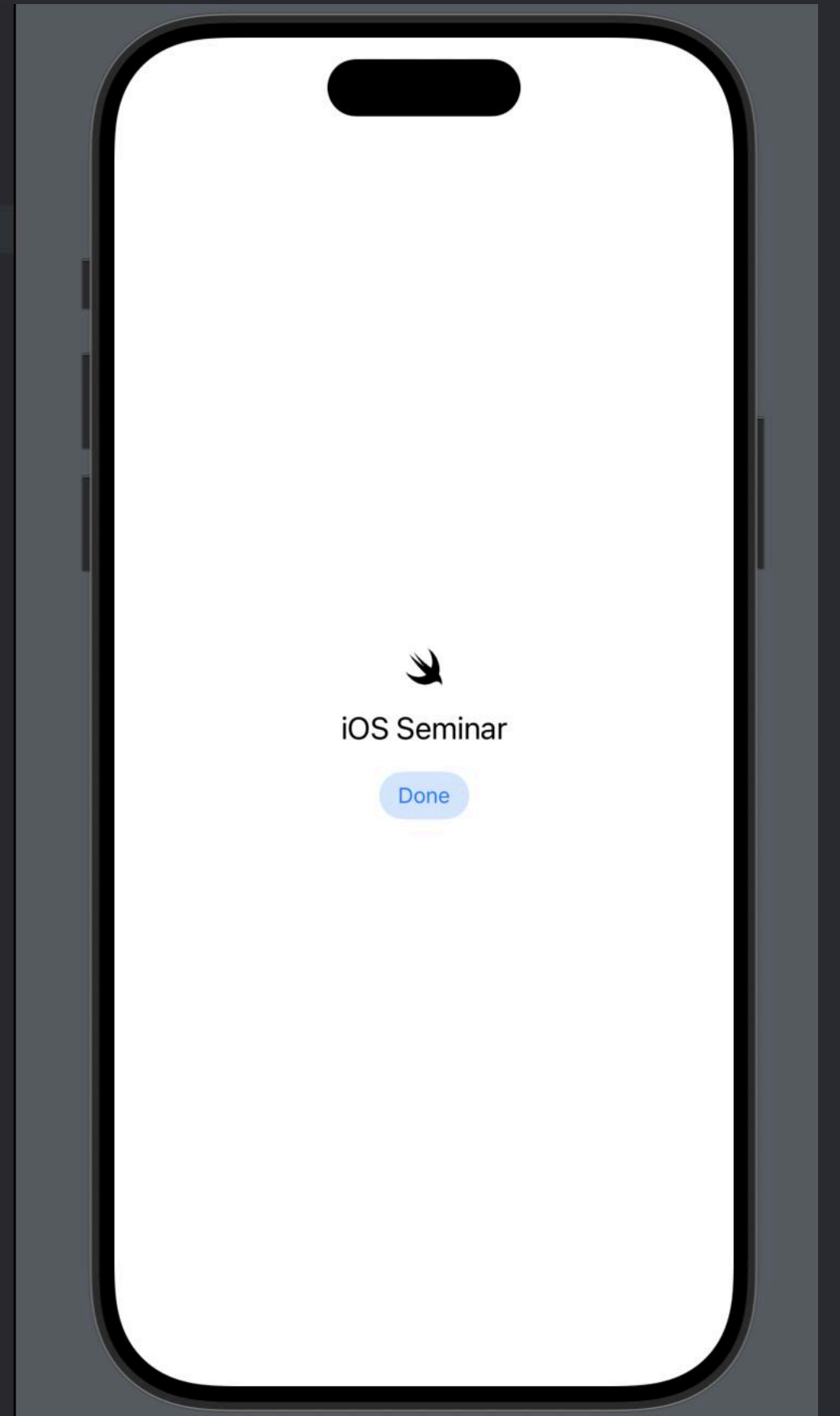
    func completeSeminar() {
        isDone = true
    }
}
```

@StateObject + @ObservedObject

```
import SwiftUI

struct PublishedExample: View {
    @ObservedObject var seminar: PublishingSeminar

    var body: some View {
        VStack(spacing: 16) {
            Spacer()
            Image(systemName: "swift")
                .resizable()
                .frame(width: 24, height: 24)
            Text(seminar.title)
                .font(.system(size: 20))
                .strikethrough(seminar.isDone)
            Button {
                seminar.completeSeminar()
            } label: {
                Text("Done")
                    .font(.system(size: 14))
            }
                .tint(.blue)
                .buttonStyle(.bordered)
                .clipShape(Capsule())
                .disabled(seminar.isDone)
            Spacer()
        }
    }
}
```



```
import SwiftUI

@main
struct Seminar4DemoApp: App {
    @StateObject var seminar = PublishingSeminar()

    var body: some Scene {
        WindowGroup {
            PublishedExample(seminar: seminar)
        }
    }
}
```

```
import Foundation
import Combine

final class PublishingSeminar: ObservableObject {
    let title = "iOS Seminar"
    @Published var isDone = false

    func completeSeminar() {
        isDone = true
    }
}
```




@StateObject

- `ObservableObject` 프로토콜을 채택하는 object 생성
- 선언하는 쪽의 lifetime 동안 단 한번 생성

@ObservedObject

- `ObservableObject`를 준수하는 타입을 subscribe
- Observable Object에서 변화가 감지되면, view를 다시 그림
- 단, @State처럼 initial value나 default 값을 넘기지 말 것

iOS 17.0+에서의 Model data Observation

- Observation(Framework) 이용
 - Observable Object와 Observer
- `@Observable` 매크로 이용
- 기본적으로, 접근 가능한 모든 property는 변화가 감지됨
 - 접근 가능한 property의 변화를 추적하고 싶지 않다면 `@ObservationIgnored` 매크로 이용

```
import Foundation

final class PublishedViewModel: ObservableObject {

    // Tracked : 새로운 Seminar 객체가 append되면 View Update
    @Published var seminar: [Seminar] = []

    // Not Tracked : session 값이 변해도 View Update ❌
    var session: Int = 0

    init() {}
}
```

PublishedViewModel.swift

```
import Observation

@Observable final class ObservableViewModel {

    // Tracked : 새로운 Seminar 객체가 append되면 View Update
    var seminar: [Seminar] = []

    // Not Tracked : session 값이 변해도 View Update ❌
    @ObservationIgnored var session: Int = 0

    init() {}
}
```

ObservableViewModel.swift

@ObservationIgnored

```
import Observation

@Observable final class ObservableSeminar {
    var title = "iOS Seminar"
    var isDone = false

    func completeSeminar() {
        isDone = true
    }

    func changeSeminarTitle() {
        title = "Spring Seminar"
    }
}
```

```
import Observation

@Observable final class ObservableSeminar {
    @ObservationIgnored var title = "iOS Seminar"
    var isDone = false

    func completeSeminar() {
        isDone = true
    }

    func changeSeminarTitle() {
        title = "Spring Seminar"
    }
}
```

ObservableSeminar.swift

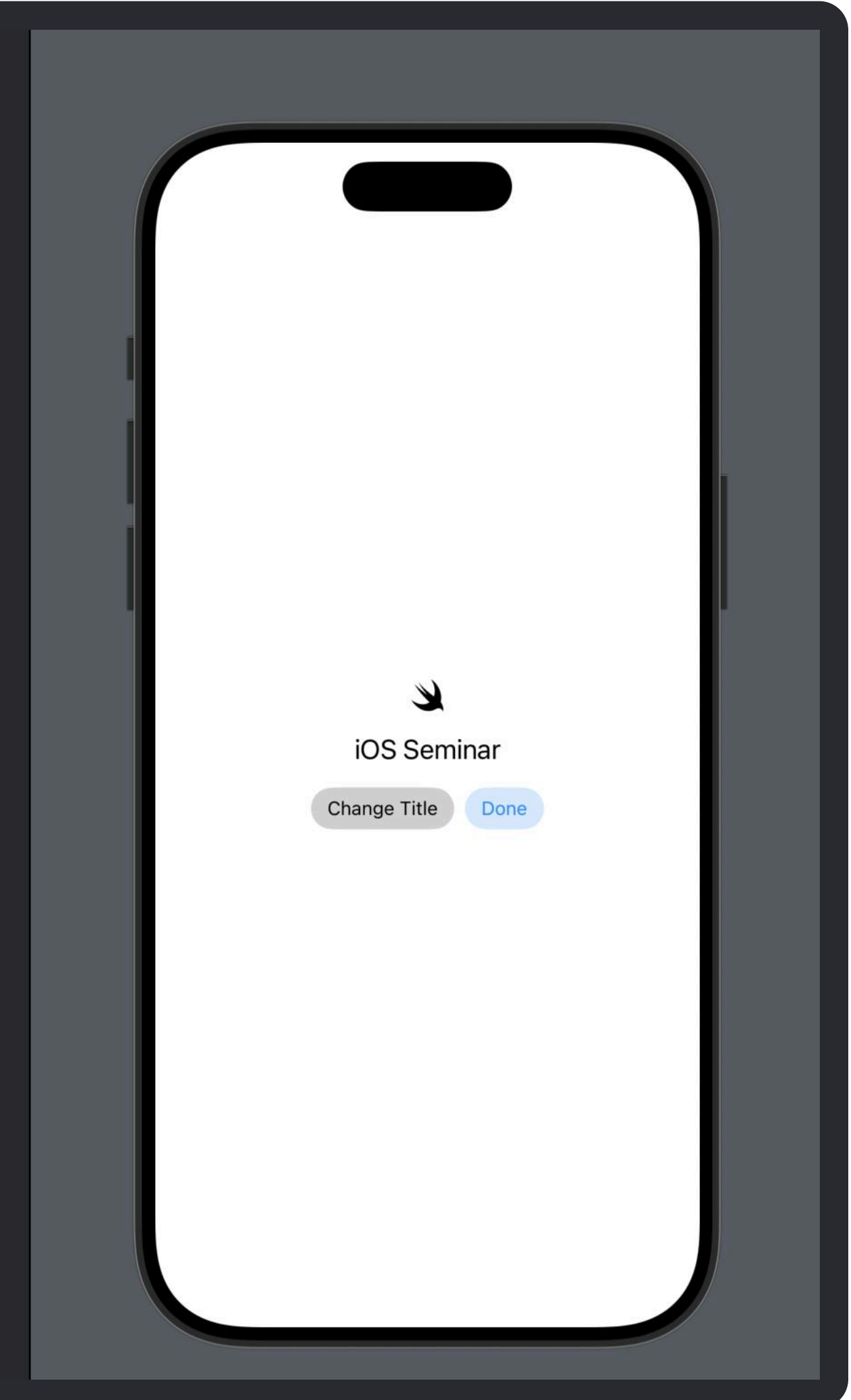
```
import SwiftUI

struct ObservableExample: View {

    @Bindable var seminar = ObservableSeminar()

    var body: some View {
        VStack(spacing: 16) {
            Image(systemName: "swift")
                .resizable()
                .frame(width: 24, height: 24)
            Text(seminar.title)
                .font(.system(size: 20))
                .strikethrough(seminar.isDone)
            HStack {
                CapsuleButton(
                    label: "Change Title",
                    tint-color: .black
                ) {
                    seminar.changeSeminarTitle()
                }
                CapsuleButton(
                    label: "Done",
                    disabled: $seminar.isDone,
                    tint-color: .blue
                ) {
                    seminar.completeSeminar()
                }
            }
                .buttonStyle(.bordered)
                .font(.system(size: 14))
        }
    }
}
```

ObservableExample.swift



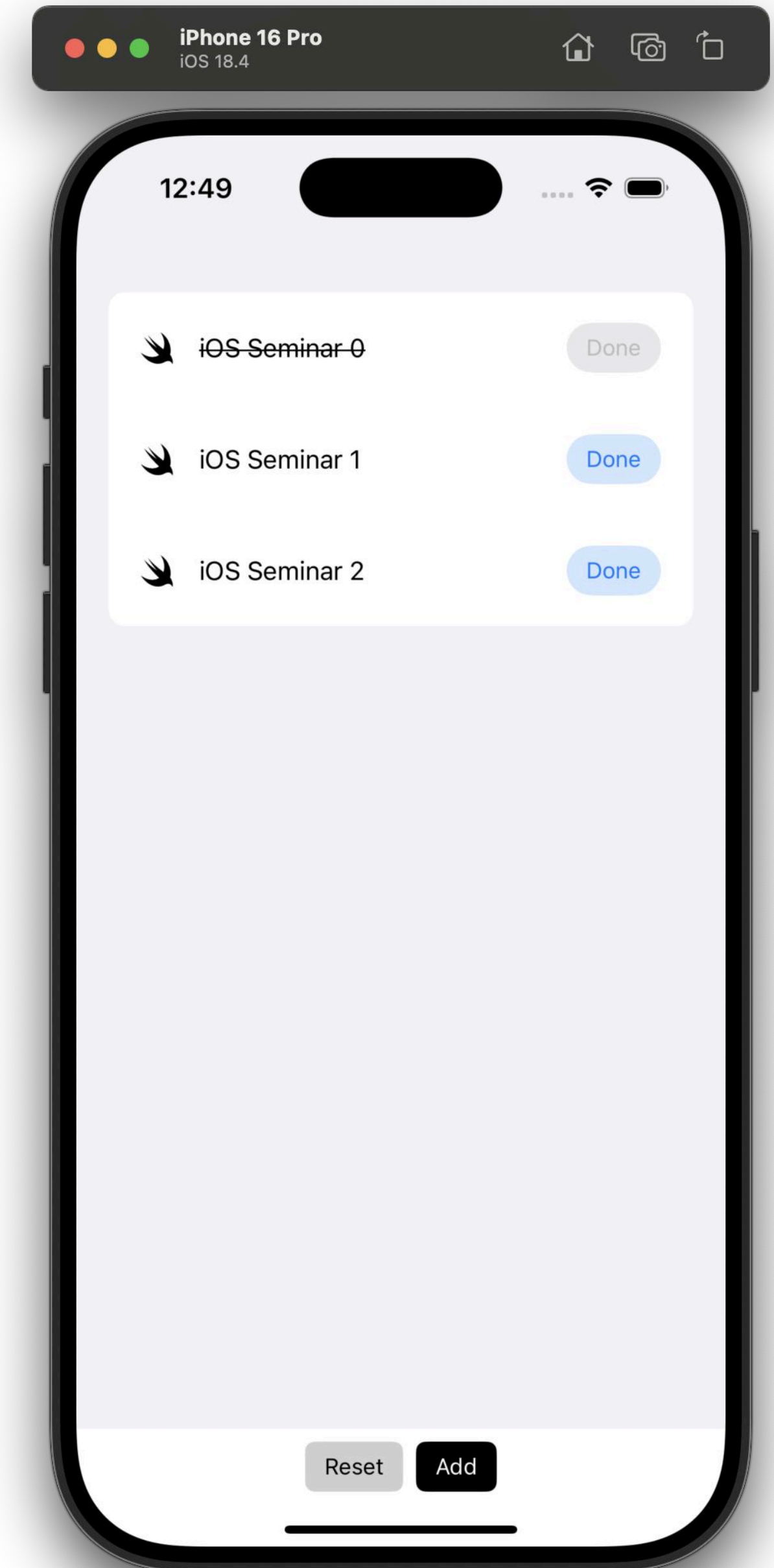
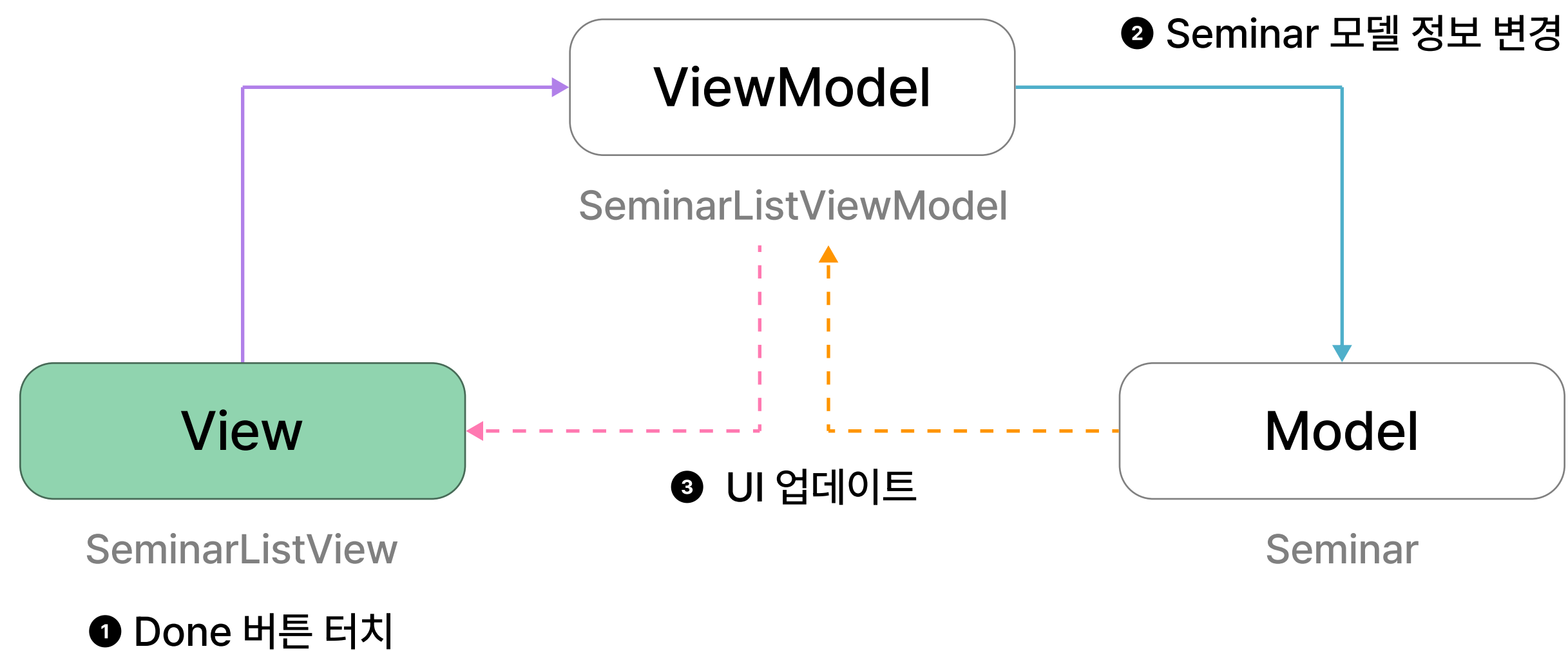
@Observable과 ObservableObject 비교

- @Observable을 이용하는 경우
 - View 업데이트: (1) Observable한 속성이 바뀌고 (2) View의 `body`가 그 속성을 직접적으로 읽는 경우
→ View가 사용하지 않는(읽지 않는) 값이 업데이트되면, View는 변하지 않는다
- ObservableObject + @Published를 이용하는 경우
 - View 업데이트: ObservableObject의 @Published가 붙은 값에서 어느 것이든 바뀌는 경우
→ View가 사용하지 않는(읽지 않는) 값이라도 업데이트가 이뤄지는 경우 View도 함께 변한다
- 관련 내용

MVVM 실습

간단한 MVVM 패턴 실습

- Seminar 목록을 보여주는 View
 - Add 버튼을 누르면 새로운 Seminar 추가
 - Reset 버튼을 누르면 "iOS Seminar 0"만 등록된 상태로 초기화
 - Done 버튼을 누르면 Seminar 완료 상태로 전환



```
import SwiftUI

struct SeminarListView: View {

    var viewModel: SeminarListViewModel

    var body: some View {
        VStack {
            List(viewModel.seminarList, id: \.session) { seminar in
                HStack(spacing: 16) {
                    swiftImage()
                    Text(seminar.title + " \ \(seminar.session)")
                        .font(.system(size: 16))
                        .strikethrough(seminar.isDone)
                    Spacer()
                    Button {
                        viewModel.complete(session: seminar.session)
                    } label: {
                        Text("Done")
                            .font(.system(size: 14))
                    }
                        .modifier(CapsuleButtonModifier())
                        .disabled(seminar.isDone)
                }
                .listRowSeparator(.hidden)
                .padding(.vertical, 8)
            }
            HStack {
                smallButton(label: "Reset", style: .bordered) {
                    viewModel.resetSeminarList()
                }
                smallButton(label: "Add", style: .borderedProminent) {
                    viewModel.addNewSeminar()
                }
            }
        }
    }
}
```



SeminarListView.swift (@Observable 버전)


```
import Foundation
import Observation

@Observable final class SeminarListViewModel {
    var seminarList: [SimpleStructSeminar] = [
        .init(session: 0)
    ]

    @ObservationIgnored private var lastSession = 0

    func addNewSeminar() {
        lastSession += 1
        seminarList.append(.init(session: lastSession))
    }

    func complete(session: Int) {
        if let idx = seminarList.firstIndex(where: { $0.session == session }) {
            seminarList[idx].completeSeminar()
        }
    }

    func resetSeminarList() {
        seminarList = [.init(session: 0)]
        lastSession = 0
    }
}
```

SeminarListViewModel.swift (@Observable 버전)

```
import Foundation

struct SimpleStructSeminar {
    var title = "iOS Seminar"
    var isDone = false
    var session: Int

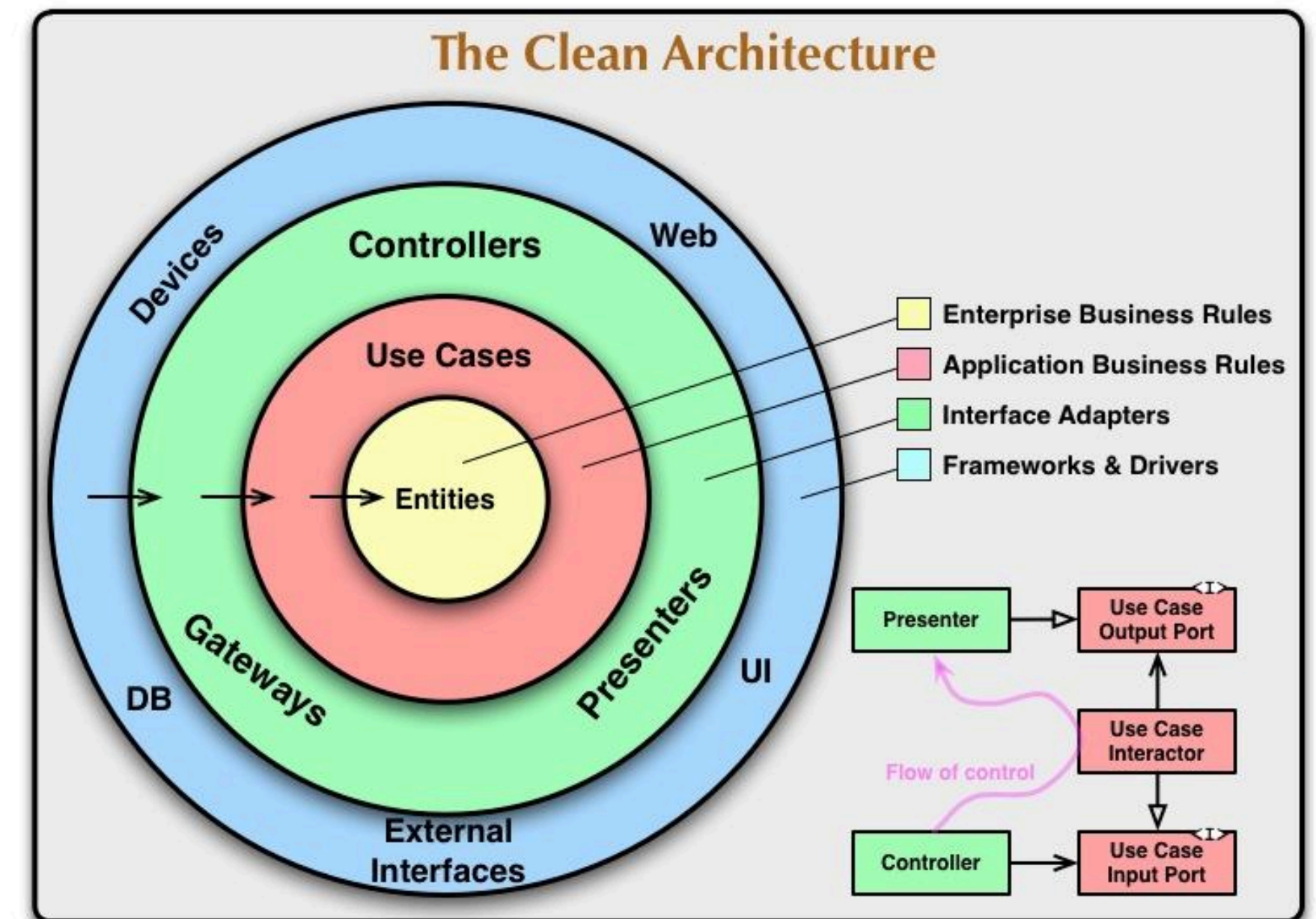
    mutating func completeSeminar() {
        self.isDone = true
    }
}
```

SimpleStructSeminar.swift

Clean Architecture

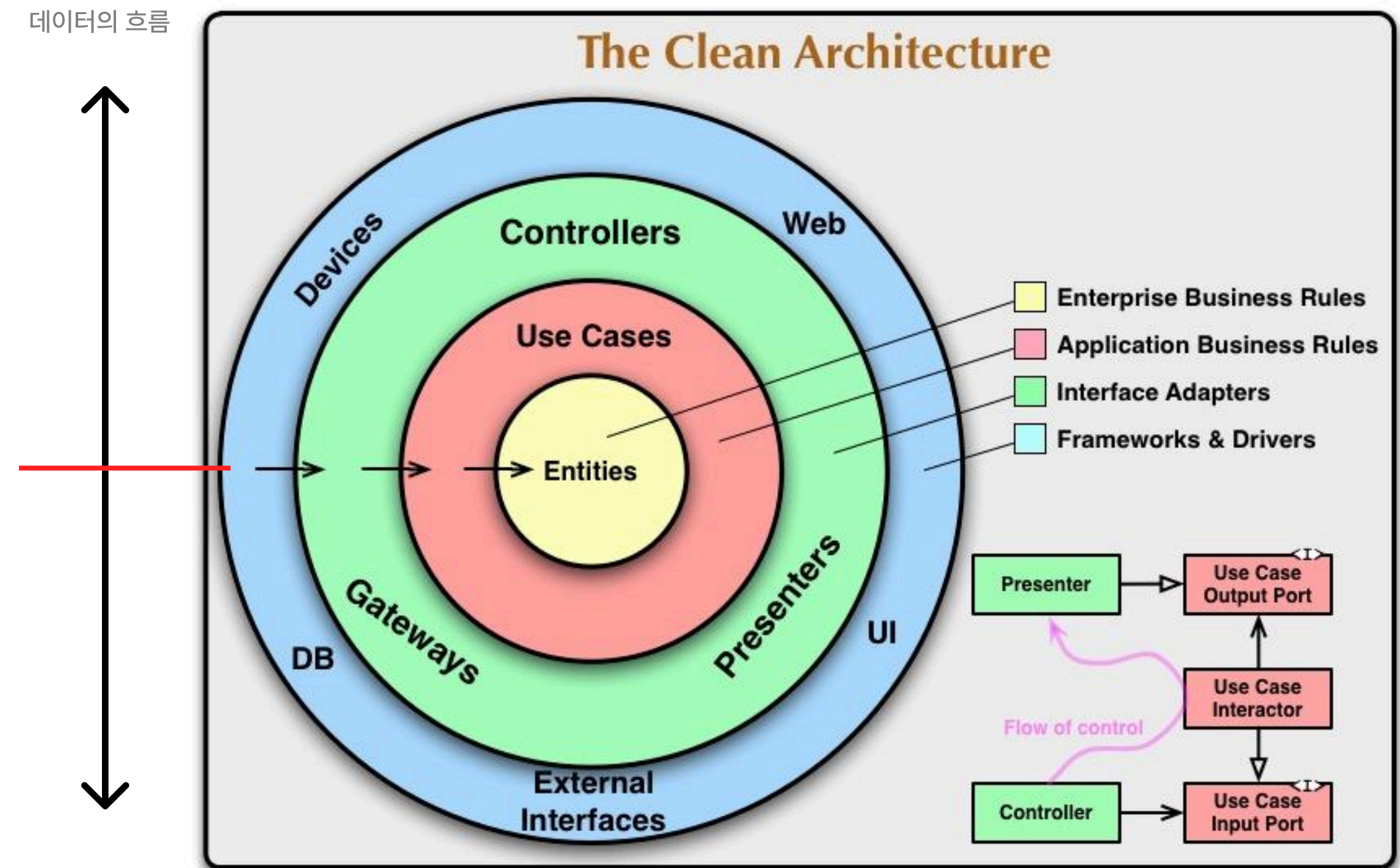
여전히 ViewModel 만으로는 부족하다

- 네트워크 통신은 누가 담당해야 할까?
- UserDefaults 등 로컬 저장소에 접근하는 역할은 누가 담당해야 할까?
- Clean Architecture
 - 이미 들어봤을 수도 있고, 들어보지 않았다면
앞으로 많이 들을 키워드
 - 책 "Clean Code"의 저자 로버트 마틴
(Robert C. Martin)이 제안한 시스템 architecture
 - SwiftUI + MVVM 참고 링크



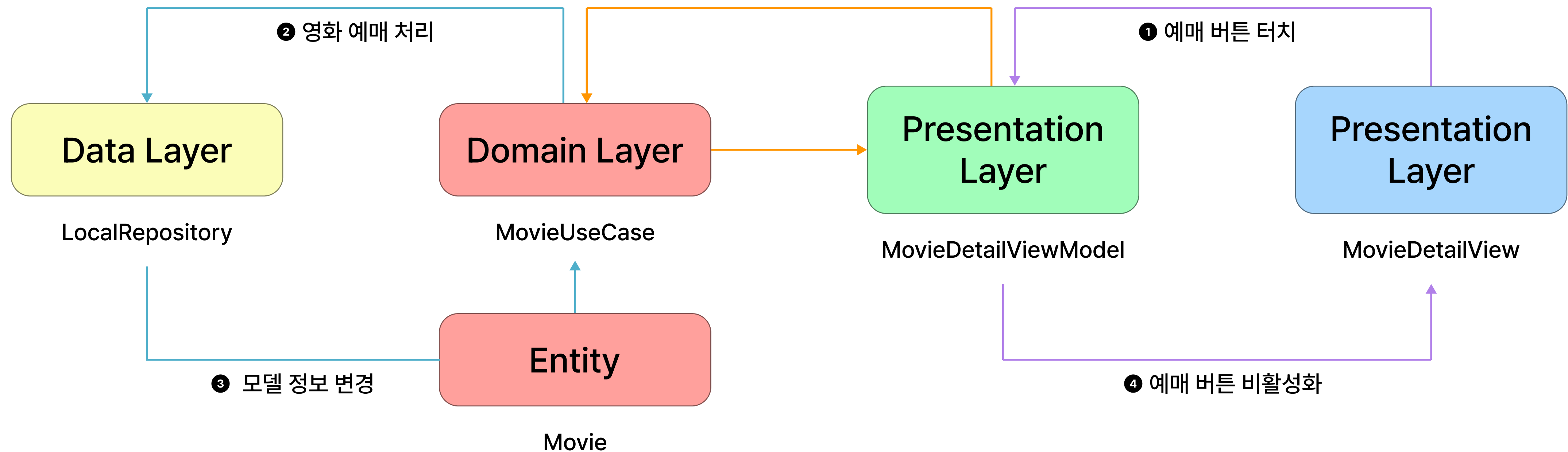
Presentation - Domain - Data

- Presentation Layer
 - View, ViewModel 등
- Domain Layer
 - Presentation Layer과 Data Layer 사이에 위치
 - 원하는 데이터를 얻기 위한 가공 로직
 - Use Case 등
- Data Layer
 - 네트워크 통신, 로컬 통신 등을 담당
 - Repository, Data Source 등



과제 2(영화 예매 앱)에 적용해 본다면?

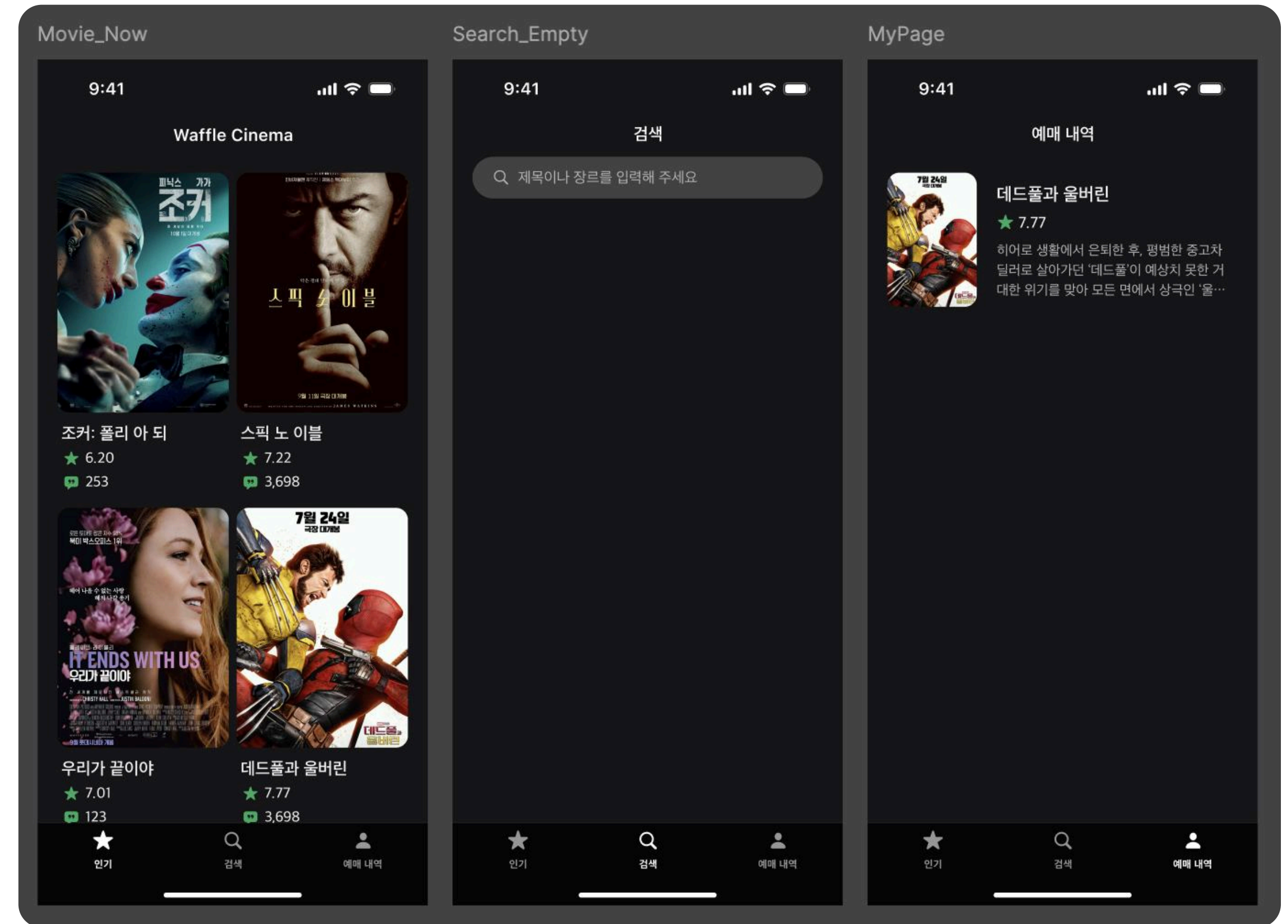
- 예매 버튼을 눌러 UserDefaults에 예매된 영화로 추가한 뒤, 예매 버튼을 비활성화하는 과정
- MovieDetailView는 LocalRepository의 존재를 모른다
→ cf. ViewController가 `movieRepository.fetchPopularMovie()`를 호출하지 않는다



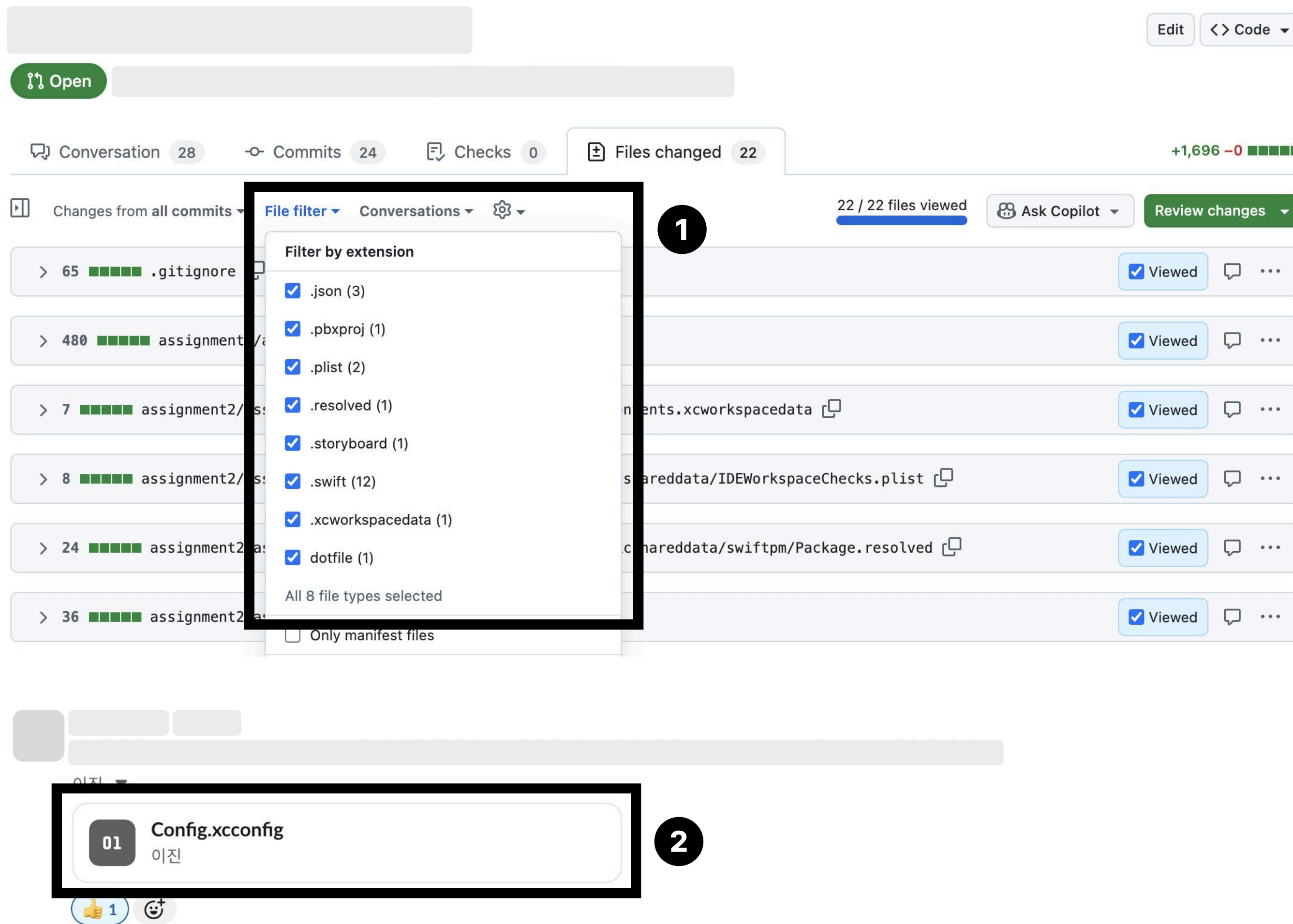
과제 4 안내

영화 예매 앱 만들기 (업그레이드)

- 과제 4 디자인 링크(Figma)
- 과제 4 스켈레톤 코드 링크(Github)
- 과제 목표
 - MVVM + 간단한 Clean Architecture 구현
 - SwiftUI를 이용한 state 관리 및 UI 구현
- 관련 키워드
 - @State, @Binding, @Environment
 - NavigationView / NavigationLink
 - UserDefaults
- 마감
 - 12월 10일 수요일 오후 11:59



과제 4 세팅



- Configuration 파일 생성 및 전달
 - `.xcconfig` 확장자를 가진 파일을 DM으로 전송
 - 과제2에서 사용한 값과 동일하거나 이미 과제 2에서 올바르게 파일을 보냈어도 다시 전송합니다
 - 과제 마감 전까지 파일을 전송하지 않을 시 스펙 구현 여부와 무관하게 탈락

과제 상세 스펙 (1)

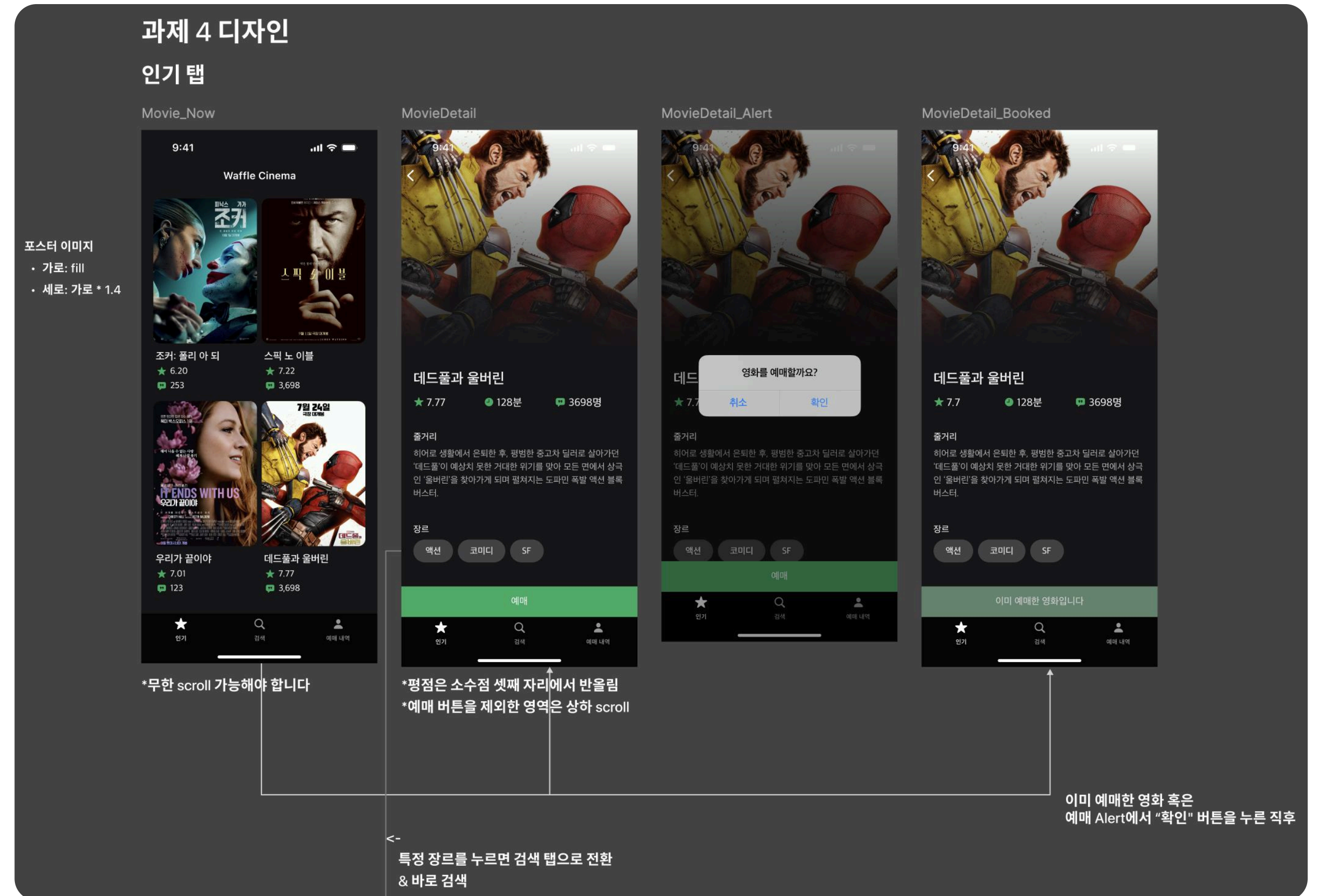
- 전체
 - Text, Color, Constraint, Corner Radius 등은 피그마에 주어진 값을 사용합니다
 - 상세 디자인 스펙은 피그마에 텍스트로 작성되어 있습니다
 - **SwiftUI + MVVM 구조로 구현합니다**
 - Clean Architecture를 필수적으로 적용할 필요는 없으나, 제공되는 스켈레톤 코드는 Clean Architecture를 가정합니다
 - 스켈레톤 코드의 사용 여부는 과제 채점에 영향을 미치지 않습니다
 - 스켈레톤 코드는 과제 구현에 최소한의 도움만을 주고 있으므로 자유롭게 수정하여 사용해 주시기 바랍니다
- Font
 - SF Pro / Apple SD Gothic Neo는 Apple의 시스템 폰트이므로 과제 진행 시 size, weight, color만 고려합니다
- Alert
 - 별도의 디자인이 필요하지 않습니다. 문구와 동작만 알맞게 설정합니다

과제 상세 스펙 (2)

• 인기 탭

- 영화 목록 조회에 사용할 API ([링크](#))
- 영화 상세정보 조회에 사용할 API ([링크](#))
- 디자인 스펙은 피그마 참고
- 장르를 터치하는 경우, 바로 검색탭으로 이동하여

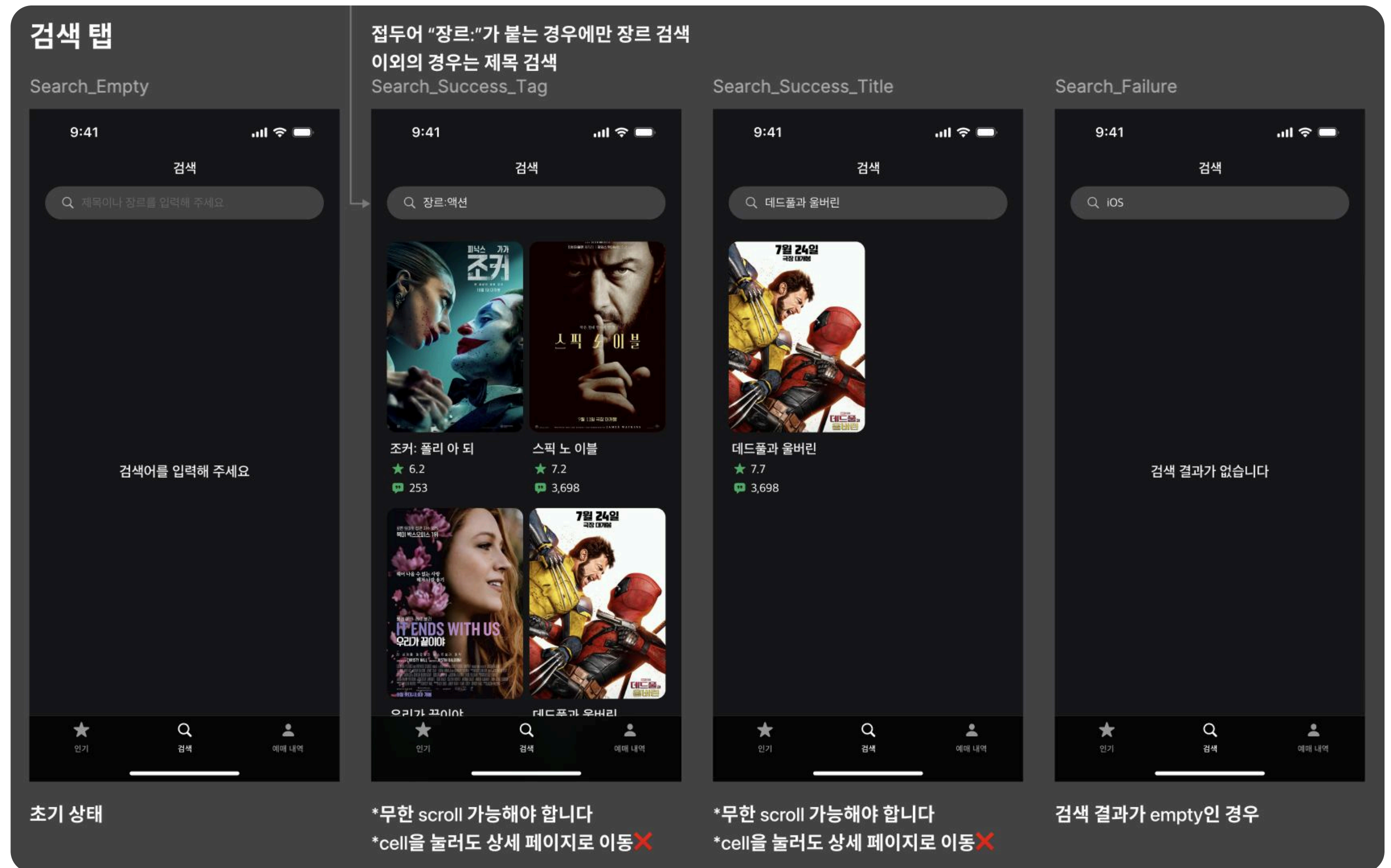
검색 기능 수행 (데모 영상 참고)



과제 상세 스펙 (3)

- 검색 탭

- 제목 검색에서 사용하는 API ([링크](#))
- 장르 검색에서 사용하는 API ([링크](#))
- 검색 수행 시 prefix 값에 주의합니다
- View의 상태에 따라 초기화면, 검색 결과 (non-Empty), 검색 결과(Empty) 각각을 알맞게 보여줍니다



과제 Bonus 스펙

- 과제 4 기본 스펙은 “예매 내역” 탭을 생성하고, 올바른 제목을 설정하는 것까지만 포함합니다
 - 기본 스펙에서 `MovieDetailView`의 예매 버튼 동작
 - 진입 시 항상 예매 전 상태 (.mainGreen)
 - 누르는 경우 alert를 띄우고, “확인”을 선택하면 예매 후 상태로 전환
- 과제 4 보너스 스펙 : 예매 내역 저장
 - 과제 2에서처럼 영화의 예매 여부를 저장하여 “예매 내역” 탭에서 보여주고, 앱을 껐다가 다시 실행하는 경우에도 유지되도록 합니다
 - 보너스 스펙에서 `MovieDetailView`의 예매 버튼 동작
 - 진입 시 예매 여부에 따라 다른 상태로 보여줍니다
 - 예매가 한번 완료되면 예매 내역에서 삭제되기 전까지는 “예매 후 상태”를 유지합니다

